Lossy Compression and Error Correction: Solutions of the Final Problem Set

Solutions of the problems in [Lossy Compression and Error Correction: Final Problem Set]({{ '/lectures/lossy_final_problemsets/' | relative_url }}). The notation is the same as in the lectures. For the computer problems we give short Python programs and the results obtained with Python 3.14, NumPy, SciPy 1.17, Pillow 12.1 and `galois` 0.4.8. With other library versions and another random seed the numbers may differ slightly, but the conclusions stay the same.

JPEG Compression

Problem 1 (DCT by hand)

(a) From the rows of the matrix C:

$$\begin{aligned} y_0 &= a\,(4+8+8+4) = 12, \\ y_1 &= b\,(4-4) + c\,(8-8) = 0, \\ y_2 &= a\,(4-8-8+4) = -4, \\ y_3 &= c\,(4-4) - b\,(8-8) = 0. \end{aligned}$$

So $y = (12, 0, -4, 0)$. Parseval's identity: $\sum x_n^2 = 16 + 64 + 64 + 16 = 160$ and $\sum y_k^2 = 144 + 16 = 160$.

(b) The basis vector $c_k[n] = \alpha_k \cos \frac{\pi(2n+1)k}{2N}$ satisfies $c_k[N-1-n] = \alpha_k \cos\left(\pi k - \frac{\pi(2n+1)k}{2N}\right) = (-1)^k c_k[n]$. So the basis vectors with even k are symmetric and those with odd k are antisymmetric (this is also visible in the matrix C: $(b, c, -c, -b)$ and $(c, -b, b, -c)$).

Claim: if $x_n = x_{N-1-n}$ for all n, then $y_k = 0$ for all odd k.

Proof: $y_k = \sum_n x_n c_k[n]$. Group the terms in pairs n and $N-1-n$: $x_n c_k[n] + x_{N-1-n} c_k[N-1-n] = x_n (c_k[n] - c_k[n]) = 0$ if k is odd (if N is odd, the middle element has $c_k[\frac{N-1}{2}] = -c_k[\frac{N-1}{2}] = 0$). $\square$

(c) $12/10 = 1.2 \to 1$, $-4/10 = -0.4 \to 0$, so $\hat{y} = (10, 0, 0, 0)$ and $x' = C^T \hat{y} = 10 \cdot (a, a, a, a) = (5, 5, 5, 5)$. The squared error is $\sum (x_n - x'_n)^2 = 1 + 9 + 9 + 1 = 20$, and $\sum (y_k - \hat{y}_k)^2 = 2^2 + 4^2 = 20$ – they agree, because $x - x' = C^T(y - \hat{y})$ and C is orthonormal. Only the average value remains in the block (and it is even slightly too small: 5 instead of 6).

(d) Every column is constant (v, v, v, v), and its DCT is $(2v, 0, 0, 0)$ (since $y_0 = a \cdot 4v = 2v$, while the other rows of C sum to 0). So CA is the matrix whose first row is $2 \cdot (4, 8, 8, 4) = (8, 16, 16, 8)$ and whose other rows are zero. Multiplying by C^T on the right transforms the rows: zero rows stay zero, and the DCT of the first row is $2y = (24, 0, -8, 0)$:

$$B = \begin{pmatrix} 24 & 0 & -8 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

The block has only horizontal frequencies (all rows are equal), so only the first row is nonzero.

(e) Subtracting the constant 128 adds $-128 \cdot \mathbf{1}$ to the block A, where $\mathbf{1}$ is the matrix of ones. It is a multiple of the DC basis image $c_0 c_0^T$ ($c_0 c_0^T = \frac{1}{8}\mathbf{1}$), so only $B_{0,0}$ changes; the other coefficients are orthogonal to a constant. $B_{0,0} = \sum_{x,y} A_{x,y} \cdot \frac{1}{8} = 8\bar{A}$, so $B_{0,0}$ decreases by $8 \cdot 128 = 1024$. Without the shift $B_{0,0} \in [0; 2040]$, with the shift $[-1024; 1016]$; the DC values are smaller and symmetric around zero. (The same happened in (d) of Problem 1: $B_{0,0} = 24 = 4 \cdot 6$, where 6 is the block average and $4 = N$.)

Problem 2 (Color spaces)

(a)

$$\begin{aligned} Y &= 0.299 \cdot 255 + 0.587 \cdot 128 + 0.114 \cdot 0 = 76.245 + 75.136 = 151.381 \to 151, \\ \mathrm{Cb} &= 128 + \tfrac{0 - 151.381}{1.772} = 128 - 85.43 = 42.57 \to 43, \\ \mathrm{Cr} &= 128 + \tfrac{255 - 151.381}{1.402} = 128 + 73.91 = 201.91 \to 202. \end{aligned}$$

(b) From $(151, 43, 202)$:

$$\begin{aligned} R &= 151 + 1.402 \cdot 74 = 254.748 \to 255, \\ B &= 151 + 1.772 \cdot (-85) = 0.380 \to 0, \\ G &= (151 - 0.299 \cdot 254.748 - 0.114 \cdot 0.380)/0.587 = 74.787/0.587 = 127.41 \to 127. \end{aligned}$$

We get $(255, 127, 0)$ instead of $(255, 128, 0)$. The matrix itself is invertible; the loss comes from rounding Y, Cb, Cr to integers (and in the computation of G the error is amplified by the division by 0.587).

(c) $(R, G, B) = (1, 0.502, 0)$:

$$\begin{aligned} Y &= 0.299 + 0.587 \cdot 0.502 = 0.594, \\ I &= 0.5959 - 0.2746 \cdot 0.502 = 0.458, \\ Q &= 0.2115 - 0.5227 \cdot 0.502 = -0.051. \end{aligned}$$

$Q \approx 0$: orange lies almost exactly on the I axis (on its positive side). This agrees with the claim in the lecture that I describes the transition from orange to blue, and Q the one from green to purple.

(d) $(Y, -I, -Q) = (0.594, -0.458, 0.051)$, and M^{-1} gives

$$\begin{aligned} R &= 0.594 - 0.956 \cdot 0.458 + 0.621 \cdot 0.051 = 0.187, \\ G &= 0.594 + 0.272 \cdot 0.458 - 0.647 \cdot 0.051 = 0.685, \\ B &= 0.594 + 1.107 \cdot 0.458 + 1.704 \cdot 0.051 = 1.187. \end{aligned}$$

This is a bluish-green tone, but $B > 1$: the color is outside the RGB cube ($\approx (48, 175, 303)$ after multiplying by 255). The YIQ parallelepiped is larger than the linear image of the RGB cube: the RGB cube is only a part of all possible (Y, I, Q). A bright blue color with such a large luma ($Y = 0.594$) is impossible, because the blue channel has a small weight (0.114) in luma. A decoder clips such a value to $B = 1$, and then Y changes too.

(e) If $R = G = B = v$, then $Y = (0.299 + 0.587 + 0.114)v = v$, so $B - Y = R - Y = 0$ and $\mathrm{Cb} = \mathrm{Cr} = 128$. The coefficients in the second and third rows of the matrix M sum to $0.5959 - 0.2746 - 0.3213 = 0$ and $0.2115 - 0.5227 + 0.3112 = 0$, so $I = Q = 0$.

(f) The Y plane takes 10^6 bytes.

Subsampling	Size of Cb and Cr	Total
4:4:4	$2 \cdot 10^6$	$3 \cdot 10^6$ bytes (24 bits per pixel)
4:2:2	$2 \cdot 500 \cdot 1000 = 10^6$	$2 \cdot 10^6$ bytes (16 bits per pixel)
4:2:0	$2 \cdot 500 \cdot 500 = 5 \cdot 10^5$	$1.5 \cdot 10^6$ bytes (12 bits per pixel)

Vision perceives luma detail much more precisely than color detail (see the pictures of the YIQ components in the lecture): edges and texture are in the Y plane, while the Cb, Cr planes are smooth. Therefore reducing the chroma resolution is almost invisible but saves half of the data.

Problem 3 (Reproduce the damage)

(a) and **(b)** Answers:

Image	Parameters	Visible damage	Changed step
A	`--block-size 32 --quality 10`	a coarse grid of squares, the smooth sky in “tiles”, ringing along the mountain edges spreads far	3 (blocks) and 5 (quantization)
B	`--keep-coefficients 3 --quality 90`	the whole image is uniformly blurred; small details (boards, fence) disappear, smooth transitions inside the blocks, a slightly visible 8×8 grid	4 (coefficient selection)
C	`--color-space rgb`	colored (red and cyan) fringes along high-contrast edges, a blurred green channel	1 (color space)

The checking program:

```
import sys
import numpy as np
from PIL import Image

a = np.asarray(Image.open(sys.argv[1]).convert("RGB"), dtype=int)
b = np.asarray(Image.open(sys.argv[2]).convert("RGB"), dtype=int)
print("largest difference:", np.abs(a - b).max())
```

```
python jpeg_lab.py barns_grand_tetons.png a.png --block-size 32 --quality 10
python compare.py a.png target-a.png        # largest difference: 0
```

(c) The grid period is 32 pixels, so `--block-size 32`. Inside the blocks only the DC and a few low-frequency coefficients remain (smooth slopes, the sky is almost one color in a block), so the quantization is coarse: `--quality` has been lowered. The exact value 10 is found by comparing the PSNR and the size with a few runs (for example, 5, 10, 20) and then checking the match.

(d) With `--color-space rgb`, the planes G and B are treated as chroma: they are subsampled 4:2:0 and quantized with the coarser chroma table. But the G plane has almost as much detail as the R plane (and G contributes 58.7% of luma), so:

- the quality is worse: subsampling and coarse quantization destroy the details of G and B, which the eye sees well (PSNR 20.56 dB instead of 26.84 dB);
- the size is larger (145.6 KiB instead of 123.3 KiB): the G and B planes have many large AC coefficients that cannot be discarded, while in YCbCr the Cb and Cr planes are smooth and compress very well. The YCbCr conversion removes the correlation between R, G, B; without it, the same information (the edges) has to be coded in three planes.

Red/pink and cyan fringes appear around the dark fence rails. The R plane (coded as luma) stays sharp, while G and B are blurred: at the edge of a dark rail, R drops sharply, but G and B only gradually. Therefore on one side there is too much G and B (cyan), on the other side too little (reddish).

(e) Parameters: `--block-size 16 --quality 5`. The width of a stair step is 16 pixels, so the block size is 16. At quality 5 the quantization steps are very large (the table is multiplied by $5000/5/100 = 10$), so in a block of a smooth gradient only the DC coefficient (the average value) remains, while the coefficient of the gentle slope (the first AC frequency) is rounded to 0. Every block becomes a single color, and the gradient turns into a staircase whose step is the block width. The thin lines in the last band disappear (the gray background remains), because their energy is in the high frequencies.

(f) A sample answer: 1-pixel-wide red text on a blue background. Both colors have a similar luma, so almost all information is in the chroma planes, which 4:2:0 subsamples – the text blurs and turns purple, even if the quantization is made finer (`--quality 95`). The damage disappears with `--chroma-subsampling 4:4:4`. Other "difficult" images: a checkerboard with 1-pixel squares (all energy in the highest frequency), sharp edges that do not coincide with block boundaries, gentle gradients at low quality (staircases, see (e)).

Other Image, Audio and Video Encodings

Problem 4 (Animated image)

(a) A clock hand that turns by 360° in 50 frames: in frame i, the angle is $\varphi_i = 360^\circ \cdot i/50$. The animation is a seamless loop if the motion is periodic and its period is a whole number of frames: frame number 50 (which no longer exists) would coincide with frame 0. So the speed must be $360^\circ/50$ per frame (or, for a sliding band, a path length divisible by the number of frames), and the last frame must not repeat the first one.

```
import io
import numpy as np
from PIL import Image

W, H, FPS, N = 320, 180, 25, 50
BG, HAND = np.array([240, 236, 224]), np.array([200, 40, 40])

def draw(i, noise=None):
    phi = 2 * np.pi * i / N                          # frame N would coincide with
 ↪  frame 0
```

```
    yy, xx = np.mgrid[0:H, 0:W].astype(float)
    cx, cy, L = W / 2, H / 2, 80
    ex, ey = np.sin(phi), -np.cos(phi)              # direction of the hand
    t = np.clip((xx - cx) * ex + (yy - cy) * ey, 0, L)
    d = np.hypot(xx - cx - t * ex, yy - cy - t * ey)  # distance to the segment
    a = np.clip(4 - d, 0, 1)[..., None]               # anti-aliased edges
    img = (1 - a) * BG + a * HAND
    if noise is not None:
        img = img + noise
    return Image.fromarray(np.clip(np.round(img), 0, 255).astype(np.uint8), "RGB")

def sizes(frames):
    out = {}
    for fmt, kw in [("GIF", {}), ("WEBP", {"quality": 80}), ("AVIF", {"quality":
     ↪  80})]:
        buf = io.BytesIO()
        frames[0].save(buf, format=fmt, save_all=True, append_images=frames[1:],
                       duration=1000 // FPS, loop=0, **kw)
        out[fmt] = buf.tell()
    return out

rng = np.random.default_rng(1)
print(sizes([draw(i) for i in range(N)]))                                    #
 ↪  (b)
print(sizes([draw(i, rng.integers(-8, 9, (H, W, 1))) for i in range(N)]))    #
 ↪  (d)
static = rng.integers(-8, 9, (H, W, 1))
print(sizes([draw(i, static) for i in range(N)]))                            #
 ↪  (e)
```

(b), **(d)**, **(e)** Results (uncompressed frames: $50 \cdot 320 \cdot 180 \cdot 3 = 8\,640\,000$ bytes):

Frames	GIF	WebP	AVIF
clean background	56.9 KB ($152:1$)	40.7 KB ($212:1$)	12.5 KB ($691:1$)
new noise in every frame	1968 KB	329 KB	474 KB
one noise pattern for all frames	130 KB	113 KB	51 KB

With a clean background AVIF is the smallest: it codes frame differences with motion compensation and an arithmetic code. GIF compresses each frame with LZW (Pillow stores only the changed rectangle around the hand), WebP uses VP8 intra coding and frame differences.

(c) A GIF frame has at most 256 colors (a palette). In a smooth color gradient, neighboring colors are mapped to the same palette color, and bands appear (*banding*). With `Dither.NONE`, every pixel is replaced with the nearest palette color – sharp bands are visible. With `Dither.FLOYDSTEINBERG`, the rounding error is diffused to the neighboring pixels (*dithering*): the correct color is obtained on average by a "pattern" of two palette colors. The bands disappear, but a fine grainy pattern appears, which moreover can change in every frame and makes the file larger.

(d) Independent noise in every pixel and frame is unpredictable: intra prediction from neighboring pixels is off by the size of the noise, and the previous frame does not help either, because it contains different noise – the

residual of a P-frame is noise with twice the variance. Noise has maximal entropy, so GIF (whose LZW code is lossless) has to code almost every bit (the size grows 35 times). The lossy codecs round away part of the noise, but their sizes still grow 8–38 times.

(e) If the noise is the same in all frames, it is part of the background: it has to be coded in the first frame (so the size is larger than with a clean background), but in the following frames the background repeats exactly, and only the motion of the hand needs to be coded. GIF gains the most ($1968 \to 130$ KB), because it is a lossless format and previously had to code all the new noise in every frame; now only a small rectangle changes. AVIF is still the smallest, because prediction from the previous frame repeats the background exactly.

Problem 5 (Sound in the frequency domain)

```
import numpy as np
from scipy.fft import dct, idct
from scipy.io import wavfile

fs = 48000
t = np.arange(2 * fs) / fs
x = sum(0.2 * np.sin(2 * np.pi * f * t) for f in [440, 440 * 2 ** (4 / 12), 440 * 2
 ↪  ** (7 / 12)])
x += 0.002 * np.sin(2 * np.pi * 470 * t)
wavfile.write("chord.wav", fs, np.round(x * 32767).astype(np.int16))

X = np.abs(np.fft.rfft(x))
freqs = np.fft.rfftfreq(len(x), 1 / fs)
db = 20 * np.log10(X / X.max() + 1e-12)          # plot db against freqs (0..2000
 ↪  Hz)

def compress(x, N, K):
    y = np.zeros_like(x)
    for s in range(0, len(x), N):
        c = dct(x[s:s + N], norm="ortho")
        c[np.argsort(np.abs(c))[:-K]] = 0         # keep the K largest
        y[s:s + N] = idct(c, norm="ortho")
    return y

def snr(x, y):
    return 10 * np.log10(np.sum(x ** 2) / np.sum((x - y) ** 2))
```

(a) The peaks are at 440 Hz (0 dB), 470 Hz (-40.6 dB, since $20\log_{10}(0.002/0.2) = -40$ dB), 554.5 Hz and 659.5 Hz. (C♯5 $= 554.37$ Hz and E5 $= 659.26$ Hz do not lie exactly on FFT frequencies, so the peak is at the nearest frequency and slightly lower: -1.1 and -3.7 dB – *spectral leakage*.) The FFT frequency resolution is $f_s/L = 1/T$: 0.5 Hz for the 2-second signal and 50 Hz for the 0.02-second fragment (960 samples). In the 2-second spectrum, 440 Hz and 470 Hz are 60 frequencies apart and easy to distinguish; in the 0.02 s fragment, both fall into neighboring frequencies (450 and 500 Hz), and the quiet tone disappears in the “side lobes” of the loud one. Short windows give good time resolution but poor frequency resolution.

(b) Results ($N = 1024$; the 470 Hz component of the error $x - x'$ is measured with a Hann window):

K	SNR	Error amplitude at 470 Hz
8	11.4 dB	0.0024

K	SNR	Error amplitude at 470 Hz
16	16.9 dB	0.0006
32	24.8 dB	0.0003
64	36.0 dB	0.00002

With $K = 8$ the error at 470 Hz is larger than the amplitude 0.002 of the tone itself – the quiet tone is effectively gone (replaced by distortion). With $K \geq 32$ it is preserved accurately. In a frame with $N = 1024$, the DCT frequency step is $f_s/(2N) = 23.4$ Hz, so 440 Hz and 470 Hz differ only by about 1.3 coefficients: the quiet tone is not a "separate" coefficient but changes the coefficients of the loud tone. Even in the original, the 470 Hz tone is practically inaudible: it is 40 dB quieter than the 440 Hz tone in the same critical band (around 500 Hz the band is about 100 Hz wide), i.e., below the masking threshold. An encoder with a psychoacoustic model would not spend any bits on it.

(c) The click at 1.0 s is sample $48\,000$. With $N = 4096$ it lies in the frame that starts at sample $45\,056$, i.e., 61.3 ms before the click; with $N = 256$ – at sample $47\,872$, i.e., 2.7 ms before the click. The error caused by the click spreads over the *whole* frame, so it starts at the beginning of the frame: 61.3 ms and 2.7 ms before the click (in the worst case, a whole frame: 85 ms and 5.3 ms). 61 ms is much more than the duration of pre-masking (5–20 ms), so in the long frame the pre-echo is audible as a "swish" before the click; in the short frame it is within pre-masking. This is why MP3 and Opus switch to short windows at transients, but use long windows for stationary signals because of their better frequency resolution.

```
xc = x.copy(); xc[48000] += 0.9
for N in [4096, 256]:
    d = np.abs(compress(xc, N, N // 16) - compress(x, N, N // 16))   # change
 ↪  caused by the click
    first = np.nonzero(d[:48000] > 0.01)[0][0]
    print(N, (48000 - first) / fs * 1000, "ms")                      # 61.3 ms, 2.7
     ↪  ms
```

(d) A 3000 Hz tone is heard (the FFT peak is at 3000 Hz). For the sample rate 8000 Hz, the Nyquist frequency is 4000 Hz; the 5000 Hz tone is above it, and its samples coincide with the samples of an $8000 - 5000 = 3000$ Hz tone: $\sin(2\pi \cdot 5000 \cdot \frac{n}{8000}) = \sin(2\pi n - 2\pi \cdot 3000 \cdot \frac{n}{8000}) = -\sin(2\pi \cdot 3000 \cdot \frac{n}{8000})$ (*aliasing*).

(e) With $q_k = q_0(1 + k/64)$ and $q_0 = 0.05$, a frame has 49 nonzero coefficients on average and $\mathrm{SNR} = 30.7$ dB; with $q_0 = 0.02$, 68 coefficients and 35.9 dB. For comparison, "the K largest" with $K = 48$ gives 31.6 dB, and with $K = 68$, 36.9 dB. In terms of SNR, quantization is slightly worse (because "the K largest" does not quantize the coefficients, but then their positions and exact values must be sent too!), but it is more realistic: quantized values are coded with few bits, and coarser quantization of the high frequencies matches the sensitivity of hearing.

Problem 6 (Theory questions about codecs)

- **(a)** *False.* Information is also lost in chroma subsampling (step 2) and in the rounding after the color space conversion (see (b) of Problem 2). Only the mathematical definition of the DCT itself is lossless; in integer arithmetic it, too, introduces small rounding errors.
- **(b)** *False.* The formula $Q^2/12$ assumes that the rounding error is uniformly distributed in $[-Q/2; Q/2]$. This is approximately correct for large coefficients. For a coefficient that is rounded to 0, the error is the coefficient y_k itself, and typical high-frequency coefficients are much smaller than $Q/2$, so the error is much smaller than $Q^2/12$. (Moreover, in JPEG each coefficient has its own $Q_{u,v}$.)

- **(c)** *False.* The number of bits for a nonzero coefficient grows roughly like $\log_2(|y|/Q)$, so a doubled step saves about one bit per coefficient and turns part of the coefficients into zeros. For example, for `jpeg_lab.py` and `barns_grand_tetons.png`: `--quality 50` gives 123.3 KiB, while `--quality 25` (all tables twice as large) gives 75.0 KiB, i.e., 1.64 times less.
- **(d)** *True.* $\mathrm{PSNR} = 10 \log_{10} \frac{255^2}{\mathrm{MSE}}$; a difference of 20 dB means that the ratio of the MSE values is $10^{20/10} = 100$.
- **(e)** *False.* The encoder searches for the best block partition, prediction mode, transform and filter parameters (*rate–distortion optimization*), while the decoder reads them from the file and executes only one variant. Decoding is much faster.
- **(f)** *False.* P-frames refer to previous I- or P-frames, B-frames to anchor frames on both sides. Playback can start only from an I-frame (a keyframe); otherwise the whole chain from the previous I-frame has to be decoded first.
- **(g)** *False.* Opus (CELT) codes only the energy of each band exactly; at a low bitrate, the shape of the high bands is filled with a "folded" spectrum of lower frequencies. Frequencies up to 20 kHz are *present* with the right loudness, but their exact values are not preserved.
- **(h)** *True.* WebM is a subset of the Matroska container (with a restricted choice of codecs), so a `.webm` file is already a valid Matroska file. Changing the extension changes nothing inside the file, and players that support MKV and VP9/Opus will play it. (The opposite direction – `.mkv` to `.webm` – works only if the file contains codecs allowed in WebM.)

Hamming Codes and Tanner Graphs

Problem 7 (The Hamming code and its extension)

(a) $x_1x_2x_3x_4 = 1011$: $y_1 = 1 \oplus 0 \oplus 1 = 0$, $y_2 = 1 \oplus 0 \oplus 1 = 0$, $y_3 = 1 \oplus 1 \oplus 1 = 1$. Codeword: $x_1 x_2 x_3 y_1 x_4 y_2 y_3 = `1010101`.

(b) `1110101` is received: $x_1 = 1, x_2 = 1, x_3 = 1, y_1 = 0, x_4 = 1, y_2 = 0, y_3 = 1$.

- $x_1 \oplus x_2 \oplus x_3 = 1 \neq y_1$, so $s_1 = 1$;
- $x_1 \oplus x_2 \oplus x_4 = 1 \neq y_2$, so $s_2 = 1$;
- $x_1 \oplus x_3 \oplus x_4 = 1 = y_3$, so $s_3 = 0$.

The syndrome $s_1s_2s_3 = 110$ is the binary index of the position x_2. In general: the bit with the index $i_1i_2i_3$ belongs to the check y_j (whose index has a one in the j-th place) if and only if $i_j = 1$. So an error in this bit changes exactly the checks with $i_j = 1$, and the syndrome is $i_1i_2i_3$. The corrected word is `1010101`, the message `1011`.

(c) `1010110` is received: $x_1x_2x_3 = 101$, $y_1 = 0$, $x_4 = 1$, $y_2 = 1$, $y_3 = 0$. Checks: $s_1 = (1 \oplus 0 \oplus 1) \oplus 0 = 0$, $s_2 = (1 \oplus 0 \oplus 1) \oplus 1 = 1$, $s_3 = (1 \oplus 1 \oplus 1) \oplus 0 = 1$. The syndrome 011 points to x_4. The decoder flips x_4 and obtains `1010010` – a valid but wrong codeword – and outputs the message `1010` (instead of `1011`). This code cannot even detect two errors.

(d) Two different codewords of the Hamming code differ in at least 3 positions. If they differ in exactly 3 (or any odd number of) positions, then the parities of their numbers of ones are different, so the parity bits y_0 differ too, and there are at least 4 differences in total. If they differ in an even number of positions, there are already at least 4.

The decoder computes the syndrome $S = s_1s_2s_3$ and the parity P of all 8 bits ($P = 0$ for a correct word):

S	P	Conclusion
000	0	no errors
$\neq 000$	1	one error at position S – correct it
000	1	one error in the bit y_0 itself
$\neq 000$	0	two errors – detect, but do not correct

One error always changes P, two errors do not. In (c): `1010101` with $y_0 = 0$ (four ones) was transmitted. `1010110` with $y_0 = 0$ is received: the number of ones is 4, $P = 0$, but $S = 011 \neq 000$ – the decoder reports two errors instead of outputting a wrong message.

(e) $y_0 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus y_1 \oplus y_2 \oplus y_3$. After substituting y_1, y_2, y_3, x_1 occurs 4 times, and x_2, x_3, x_4 occur 3 times each. Modulo 2 this leaves $y_0 = x_2 \oplus x_3 \oplus x_4$.

Problem 8 (A Tanner graph and erasures)

(a)

$$H = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

The Tanner graph has 8 bit nodes and 4 check nodes; each of $b_1, \dots, b_4$ is connected to 3 checks, each of $b_5, \dots, b_8$ to one (b_{4+i} to p_i). Each check p_i contains exactly one of the bits $b_5, \dots, b_8$, so it can be expressed: $b_5 = b_1 \oplus b_2 \oplus b_3$, $b_6 = b_1 \oplus b_2 \oplus b_4$, $b_7 = b_1 \oplus b_3 \oplus b_4$, $b_8 = b_2 \oplus b_3 \oplus b_4$. So $b_1, \dots, b_4$ are free, and the code has $2^4 = 16$ codewords. `1011` gives $b_5 = 0, b_6 = 0, b_7 = 1, b_8 = 0$, i.e., `10110010`.

This is the extended Hamming code of Problem 7: b_5, b_6, b_7 are y_1, y_2, y_3, and $b_8 = x_2 \oplus x_3 \oplus x_4 = y_0$ ((e) of Problem 7). Only the order of the bits is different.

(b) Known: $b_3 = 1, b_4 = 1, b_6 = 0, b_7 = 1, b_8 = 0$.

1. p_3 (b_1, b_3, b_4, b_7) contains only one unknown: $b_1 = b_3 \oplus b_4 \oplus b_7 = 1$.
2. p_4 (b_2, b_3, b_4, b_8): $b_2 = 1 \oplus 1 \oplus 0 = 0$.
3. p_1 (b_1, b_2, b_3, b_5): $b_5 = 1 \oplus 0 \oplus 1 = 0$.

Check with p_2: $b_1 \oplus b_2 \oplus b_4 \oplus b_6 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$. (Initially p_1 and p_2 contain two unknowns each, so one has to start with p_3 or p_4.)

(c) Each check contains three of the bits $b_1, \dots, b_4$, i.e., three unknowns – no check can be used, and peeling stops immediately. (A set of erasures in which every check contains either zero or at least two erased bits is called a *stopping set*.) Nevertheless, the system

$$\begin{aligned} b_1 \oplus b_2 \oplus b_3 &= b_5 = 0, \\ b_1 \oplus b_2 \oplus b_4 &= b_6 = 0, \\ b_1 \oplus b_3 \oplus b_4 &= b_7 = 1, \\ b_2 \oplus b_3 \oplus b_4 &= b_8 = 0 \end{aligned}$$

has a unique solution. Adding all the equations, each b_i occurs three times: $b_1 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$. Adding this to each equation: $b_4 = 1 \oplus 0 = 1$, $b_3 = 1 \oplus 0 = 1$, $b_2 = 1 \oplus 1 = 0$, $b_1 = 1 \oplus 0 = 1$. `1011` is recovered – Gaussian elimination (or maximum likelihood decoding) can do more than peeling.

(d) If two codewords agree in all non-erased positions, the decoder cannot distinguish them. Their difference (XOR) is a codeword whose ones are only in erased positions. The code is linear and its minimum distance is 4, so a nonzero codeword has at least 4 ones: any 3 erasures can be recovered. A set of four erasures cannot be recovered if and only if it is the set of positions of a codeword with four ones. For example, the message `1000` gives `10001110`, so the erasures $\{b_1, b_5, b_6, b_7\}$ cannot be recovered: the transmitted word and the word with `10001110` added to it cannot be distinguished.

(e)

```python
import numpy as np

def peel(checks, received):
    """checks: a list of checks (lists of bit indices); received: 0, 1 or None."""
    b = list(received)
    progress = True
    while progress and None in b:
        progress = False
        for chk in checks:
            unknown = [j for j in chk if b[j] is None]
            if len(unknown) == 1:
                b[unknown[0]] = sum(b[j] for j in chk if b[j] is not None) % 2
                progress = True
    return b

def random_ldpc(n, dv=3, dc=6, rng=None):
    """A (3, 6)-regular Tanner graph: shuffle the n*dv "ends" of the edges in
     ↪ random order."""
    sockets = rng.permutation(np.repeat(np.arange(n), dv))
    return [list(sockets[i * dc:(i + 1) * dc]) for i in range(n * dv // dc)]

rng = np.random.default_rng(2026)
n = 1200
checks = random_ldpc(n, rng=rng)
for eps in [0.30, 0.35, 0.40, 0.45, 0.50]:
    ok = sum(None not in peel(checks, [None if rng.random() < eps else 0 for _ in
 ↪  range(n)])
             for _ in range(100))
    print(eps, ok)
```

Results (the number of successful decodings out of 100):

ε	0.30	0.35	0.40	0.42	0.45	0.50
successful	100	100	92	57	8	0

It is enough to test the all-zero codeword: the course of the peeling decoder (which checks contain one unknown at which moment) depends only on the positions of the erasures, not on the values of the bits. If the codeword is c, then all recovered values are the bits of c exactly when they are zeros for the all-zero word (the code is linear: XOR with c turns one case into the other). Decoding deteriorates sharply around $\varepsilon \approx 0.42$–0.43, which

agrees with the theoretical threshold 0.429. For a longer code the transition would be even sharper. (In the random graph, a check may contain the same bit twice; this is rare and does not affect the result much.)

Reed–Solomon Codes and Finite Fields

Problem 9 (A Reed–Solomon code over GF(7))

(a) $1 \cdot 1 = 1, 2 \cdot 4 = 8 \equiv 1, 3 \cdot 5 = 15 \equiv 1, 6 \cdot 6 = 36 \equiv 1$:

a	1	2	3	4	5	6
a^{-1}	1	4	5	2	3	6

(b) $f(x) = 3x + 5$: $f(0) = 5$, $f(1) = 8 \equiv 1$, $f(2) = 11 \equiv 4$, $f(3) = 14 \equiv 0$, $f(4) = 17 \equiv 3$. The values $5, 1, 4, 0, 3$ are sent.

(c) Any $k = 2$ correct values suffice, so $s - k = 3$ erasures can be recovered. Errors: $c \le (s-k)/2 = 1.5$, i.e., 1 error. The arguments must be different field elements, so $s \le 7$; with $s = 7$, $(7-2)/2 = 2.5$, i.e., 2 errors (or 5 erasures) can be corrected.

(d) We know $f(1) = 1$ and $f(4) = 3$. The Lagrange basis polynomials:

$$L_1(x) = \frac{x-4}{1-4} = (x-4)\cdot(-3)^{-1}, \qquad L_4(x) = \frac{x-1}{4-1} = (x-1)\cdot 3^{-1}.$$

Modulo 7: $-3 \equiv 4$, $4^{-1} = 2$, so $L_1(x) = 2(x-4) = 2x-8 \equiv 2x+6$; $3^{-1} = 5$, so $L_4(x) = 5(x-1) = 5x-5 \equiv 5x+2$. Then

$$f(x) = 1 \cdot (2x+6) + 3 \cdot (5x+2) = 17x + 12 \equiv 3x + 5.$$

The message is $3, 5$ (check: $f(1) = 8 \equiv 1$, $f(4) = 17 \equiv 3$).

(e) $k = 2$, $c = 1$: $Y(x) = x + y_0$, $Z(x) = z_2x^2 + z_1x + z_0$, and the equations $Z(i) = (i + y_0) \cdot r_i$ modulo 7 ($3^2 = 9 \equiv 2$, $4^2 = 16 \equiv 2$):

$$\begin{aligned}
i = 0: &\quad z_0 &&\equiv 5y_0,\\
i = 1: &\quad z_2 + z_1 + z_0 &&\equiv 1 + y_0,\\
i = 2: &\quad 4z_2 + 2z_1 + z_0 &&\equiv 6(2 + y_0) \equiv 5 + 6y_0,\\
i = 3: &\quad 2z_2 + 3z_1 + z_0 &&\equiv 0,\\
i = 4: &\quad 2z_2 + 4z_1 + z_0 &&\equiv 3(4 + y_0) \equiv 5 + 3y_0.
\end{aligned}$$

- Subtracting the equation $i = 3$ from the equation $i = 4$: $z_1 \equiv 5 + 3y_0$.
- Substituting $z_0 = 5y_0$ and z_1 into the equation $i = 3$: $2z_2 + 15 + 9y_0 + 5y_0 = 2z_2 + 1 + 0 \cdot y_0 \equiv 0$ (since $14y_0 \equiv 0$), so $2z_2 \equiv 6$ and $z_2 \equiv 3$.
- The equation $i = 1$: $3 + 5 + 3y_0 + 5y_0 = 8 + 8y_0 \equiv 1 + y_0$ – holds for every y_0 (gives no new information).
- The equation $i = 2$: $12 + 10 + 6y_0 + 5y_0 = 22 + 11y_0 \equiv 1 + 4y_0 \equiv 5 + 6y_0$, so $-2y_0 \equiv 4$, $5y_0 \equiv 4$, $y_0 \equiv 4 \cdot 5^{-1} = 4 \cdot 3 = 12 \equiv 5$.
- Then $z_1 \equiv 5 + 15 = 20 \equiv 6$ and $z_0 \equiv 25 \equiv 4$.

$Y(x) = x + 5$, its root is $x = 2$ ($2 + 5 = 7 \equiv 0$): the value r_2 is wrong. $Z(x) = 3x^2 + 6x + 4$. Divide by $Y(x)$:

$$\begin{aligned} 3x^2 + 6x + 4 - 3x(x + 5) &= -9x + 4 \equiv 5x + 4, \\ 5x + 4 - 5(x + 5) &= -21 \equiv 0. \end{aligned}$$

$f(x) = 3x + 5$ with no remainder: the message is $3, 5$, and the correct value at position 2 is $f(2) = 4$ (not 6).

(f) The line through $(0, 5)$ and $(1, 1)$: $f(x) = 5 + (1 - 5)x \equiv 3x + 5$. It also passes through $(3, 0)$ and $(4, 3)$, but not through $(2, 6)$ ($f(2) = 4$). So four points lie on one line, and it is the only one: if two lines each passed through four of the five points, they would have at least 3 common points, but different lines have at most one common point. An exhaustive search must test all $\binom{s}{k}$ sets of points; for $s = 255$, $k = 223$ there are about 10^{40} sets, while the Berlekamp–Welch method takes $O(s^3)$ operations.

Problem 10 (The field GF(8) and Reed–Solomon codes for bytes)

(a) Each next power is obtained by multiplying by x and replacing $x^3 = x + 1$:

i	0	1	2	3	4	5	6	7
α^i	1	x	x^2	$x+1$	x^2+x	x^2+x+1	x^2+1	1
bits	`001`	`010`	`100`	`011`	`110`	`111`	`101`	`001`

For example, $\alpha^5 = x \cdot (x^2 + x) = x^3 + x^2 = x^2 + x + 1$ and $\alpha^6 = x^3 + x^2 + x = x^2 + 1$. Among $\alpha^0, \ldots, \alpha^6$ are all seven nonzero bit strings, so α is a primitive element, and $\alpha^7 = 1$.

(b) $x^2+1 = \alpha^6, x+1 = \alpha^3$, so $(x^2+1)(x+1) = \alpha^9 = \alpha^2 = x^2$. $x^2+x = \alpha^4$, so $(x^2+x)^{-1} = \alpha^{-4} = \alpha^3 = x+1$. In the table of the lecture: row $x^2 + 1$, column $x + 1$ – x^2; row $x^2 + x$, column $x + 1$ – 1. When polynomials are added, the corresponding coefficients are added modulo 2, and $0 \oplus 0 = 0, 0 \oplus 1 = 1, 1 \oplus 1 = 0$ – this is the XOR of each bit (there are no carries).

(c) $f(x) = \alpha x + 1$:

- $f(0) = 1 = \$ 001$;
- $f(1) = \alpha + 1 = \$ 011$;
- $f(\alpha) = \alpha^2 + 1 = \$ 101$;
- $f(\alpha^2) = \alpha^3 + 1 = (x + 1) + 1 = x = \$ 010$.

The message has $2 \cdot 3 = 6$ bits, the codeword $4 \cdot 3 = 12$ bits. The code corrects $(4 - 2)/2 = 1$ erroneous symbol (up to 3 bits within one symbol).

(d)

```
import numpy as np
import galois

GF8 = galois.GF(2**3)                    # p(x) = x^3 + x + 1
a = GF8.primitive_element                # alpha = x = 2
print([format(int(a**i), "03b") for i in range(8)])
print(GF8(5) * GF8(3), GF8(6) ** -1)     # x^2 = 4, x + 1 = 3

def points(GF, s):
```

```
    """The arguments 0, 1, alpha, alpha^2, ..., alpha^(s-2)."""
    al = GF.primitive_element
    return GF([0] + [int(al**i) for i in range(s - 1)])

def rs_encode(msg, s):
    return galois.Poly(msg)(points(type(msg), s))   # msg = a_1, ..., a_k

def berlekamp_welch(r, k):
    GF = type(r); s = len(r); c = (s - k) // 2
    xs = points(GF, s)
    # unknowns z_0..z_{k+c-1}, y_0..y_{c-1}:  Z(x_i) - r_i (y_0 + ... ) = r_i x_i^c
    A = GF(np.zeros((s, k + 2 * c), dtype=int))
    for j in range(k + c):
        A[:, j] = xs ** j
    for j in range(c):
        A[:, k + c + j] = -r * xs ** j
    M = np.concatenate([A, (r * xs ** c)[:, None]], axis=1).row_reduce()
    sol = GF(np.zeros(k + 2 * c, dtype=int))
    for row in M:                                   # free variables = 0
        nz = np.nonzero(row[:-1])[0]
        if len(nz) == 0:
            if row[-1] != 0:
                return None                         # the system has no solution
            continue
        sol[nz[0]] = row[-1]
    Z = galois.Poly(sol[:k + c][::-1])
    Y = galois.Poly(np.concatenate([GF([1]), sol[k + c:][::-1]]))
    q, rem = divmod(Z, Y)
    return q if rem == 0 else None
```

The system is underdetermined if there are fewer than c errors, so `np.linalg.solve` (which needs a square, invertible matrix) does not work; `row_reduce` (Gaussian elimination) with the free variables set to 0 gives one of the solutions, and by the claim in the lecture all solutions give the same Z/Y. Over $\mathrm{GF}(2^8)$ with $k = 8$, $s = 16$, $c = 4$ errors can be corrected. In 200 random trials: 3 and 4 errors were corrected in all 200 cases, 5 errors in none (the system has no solution or the division leaves a remainder; in rare cases one could also obtain another, wrong codeword).

(e)

```
rs = galois.ReedSolomon(255, 223)
text = "Algoritmi telekomunikacijas un drosibas risinajumos".encode()
m = rs.field(np.frombuffer(text.ljust(223, b" "), dtype=np.uint8))
c = rs.encode(m)
rng = np.random.default_rng(7)
for L in [16, 17]:
    e = c.copy()
    e[40:40 + L] ^= rs.field(rng.integers(1, 256, L))     # L consecutive corrupted
    ↪ bytes
    d, nerr = rs.decode(e, errors=True)
    print(L, nerr, bytes(np.array(d, dtype=np.uint8))[:len(text)] == text)
```

With 16 corrupted bytes the decoder returns the correct message and the number of corrected errors 16; with 17, the number of errors -1 (decoding failed), and the returned bytes do not match the original message. The code has $255 - 223 = 32$ check bytes, so it corrects up to 16 erroneous bytes. If there are more errors, the decoder usually detects this, but it may also (very rarely) find a wrong codeword.

(f) 128 consecutive corrupted bits cover $\lceil 128/7 \rceil = 19$ or 20 Hamming blocks. The fully hit blocks contain several errors (with random values, on average in half of the 7 bits), which the Hamming code cannot correct – at best only the boundary blocks with a single error are corrected. So practically all 128 bits remain wrong (and the decoder "corrects" some other bits as well). For the Reed–Solomon code a symbol is a whole byte: 128 consecutive corrupted bits are only 16 (or 17, if they do not align with byte boundaries) erroneous symbols, and they are corrected. In an error burst, what matters is not the number of erroneous bits but the number of erroneous *symbols*. Compact discs additionally use *interleaving* (CIRC): the symbols of one code group are spread over a long stretch of the disc, so a scratch damages only a few symbols in each of many groups.